

Interactive graphics final project

CyberNeon Claw

Documentation and analysis

Luca Adamo 1987858

12/07/2026

1.	INTRODUCTION	1
2.	USER MANUAL	1
2.1.	WORKSPACE CAMERA NAVIGATION (ORBIT CONTROLS)	1
2.2.	ARM KINEMATICS AND MOVEMENT CONTROLS	2
2.3.	CONFIGURATION PANEL AND INSTRUCTIONS OVERLAY	2
3.	LIBRARIES	2
4.	TECHNICAL ANALYSIS	3
4.1.	LIGHTING SETUP	3
4.2.	MODELS	3
4.3.	MATERIAL AND TEXTURES	4
4.4.	ANIMATIONS	5
5.	PROJECTUAL CHOICES	6
5.1.	PROCEDURAL VS. IMPORTED 3D MODELS	6
5.2.	CUSTOM JAVASCRIPT VS. EXTERNAL PHYSICS ENGINE	6
5.3.	IMPLEMENTING OPTIONAL ADVANCED LIGHTING AND REAL-TIME SHADOWS	7
5.4.	USER INTERACTION AND UI CUSTOMIZATION (THEMES AND DIFFICULTY)	7
6.	PROJECT ARCHITECTURE AND FILES	8

1.Introduction

This document presents the technical documentation for **CyberNeon Claw**, a 3D interactive graphics application developed as the final project for the **Interactive Graphics** course.

The main gameplay idea is inspired by traditional arcade claw machines, modified into a dynamic version where the user controls a robotic arm to catch moving targets.

The project consists of a real-time simulation of a robotic arm acting inside a dynamic workspace. The primary goal of the user is to control the manipulator's joints to intercept, grab, and collect moving target spheres to score as many points as possible before the countdown timer expires.

The application includes an overlay Head-Up Display that allows the user to start the game, switch between three distinct aesthetic profiles (Cyberpunk, Industrial, and Retrowave), and select the gameplay difficulty. To challenge the player, the game implements a progressive scaling system distributed across three difficulty settings:

- **Easy:** The simulation starts with three target balls. As the player scores points, the linear speed of the spheres increases, their radius scales down, and the number of concurrent targets on the board progressively decreases until only one remains.
- **Medium:** Following the same progressive scaling logic, this mode initiates the game with two balls, which are natively smaller and faster compared to the easy preset.
- **Hard:** Designed for maximum challenge, this mode begins directly with a single target sphere that features a reduced radius, a higher initial translation speed and the time decreasing every time the user catches a ball.

2.User manual

The interaction with the 3D simulation is managed directly via keyboard controls for the robotic arm joints and mouse movements for the viewport camera navigation, with the instructions on the overlay display.

2.1. *Workspace Camera Navigation (Orbit Controls)*

The user can navigate around the scene using the mouse pointer via the integrated OrbitControls module:

- **Left click + Drag:** Rotates the camera viewpoint around the center of the scene.
- **Mouse wheel:** Zooms the camera in or out (bounded by software constraints to prevent intersection with objects).
- **Right click + Drag:** Moves the camera viewport orientation (pan).

2.2. Arm Kinematics and Movement Controls

The movement of the robotic arm joints is mapped directly to the keyboard, allowing the user to control individual rotational degrees of freedom along the kinematic chain:

- **Left Arrow / Right Arrow:** Rotate the robot's base on the horizontal plane with an Y-axis rotation.
- **Up Arrow / Down Arrow:** Control the mechanical tilt and pitch angle of the lower arm with a Z-axis rotation.
- **W / S:** Control the mechanical tilt and pitch angle of the upper arm (Z-axis rotation).

2.3. Configuration panel and instructions overlay

The 2D interface is split into an initial informational overlay and a configuration sidebar to manage the simulation state:

- **Instruction Overlay:** Appears automatically when the application loads. It provides the user an introductory overview of the game mechanics and details about the necessary keyboard shortcuts required to operate the robotic arm in order to be able to know how to play the game.
- **Sidebar Menu:** Contains the primary gameplay controls:
 - **Start Button:** Triggers the gameplay state, starts the 30-second timer, and begins the procedural target spawning loop.
 - **Stop Button:** It shows when the game starts, and allow the user to quit the game without having to wait for the countdown to end.
 - **Difficulty Selector:** The radio buttons that offers to the user the possibility to choose the difficulty.
 - **Theme Selector:** Executes a live reload of the scene assets. Switching between *Cyberpunk*, *Industrial*, and *Retrowave* instantly swaps the material configurations on the meshes and updates the emissive colors of the environment's LED lighting setups.

3. Libraries

In the project I used the following external tools and libraries to manage the graphics and animations without writing low-level code:

- **Three.js:** The main library used to build the 3D environment. It handles the creation of objects, lighting, textures, and the rendering of the scene in the browser. It was useful because it allowed me to build the game board, the moving spheres, and the robot

parts using simple built-in shapes (like cylinders and boxes), and to quickly load and apply the textures.

- **OrbitControls.js:** A plugin for Three.js that allows the user to move the camera with the mouse. It manages the rotation, zooming, and panning of the view.
- **Tween.js:** A simple animation tool that allows to make the animations smoother. It is used to make the robotic claw open and close smoothly rather than snapping instantly. It was helpful because I could just feed it with the starting angle and the closed angle, and the library automatically calculated the smooth frame-by-frame interpolation.

4. Technical analysis

This section takes a closer look at the entire scene, starting with the lights, moving on to the textures, and ending with the animations, including a description of everything that makes up the project and how it was created.

4.1. *Lighting setup*

The 3D environment uses a mix of different light sources to give depth to the room and match the style of each theme:

- **Ambient Light:** This provides a soft, baseline illumination for the whole room, ensuring that the unlit sides of the objects are still slightly visible.
- **Directional Light:** Placed above the table, this acts as the main light source. It is configured to cast real-time shadows of the moving robotic arm and the balls onto the table surface.
- **Point Lights:** These are small, localized lights placed around the table edges and on the strips starting from the corners. They simulate glowing neon LEDs, and their colors change dynamically whenever the user selects a new theme.

4.2. *Models*

The hierarchical model required for the project is obviously the mechanical arm. I choose to build by myself using the forms provided by the library three, making all of them procedurals, instead of taking any already made 3d model. I tried to make the robotic arm as complex as possible to give it the realistic and articulated aspect that a pre-made 3D model would have had, while still maintaining the ease of managing a procedural model, rather than having to study an entire hierarchy that I didn't create myself. Instead of positioning every part independently in the 3D space, each piece is attached to the previous one in a specific order:

- **Base:** A multi-tiered cylindrical foundation consisting of 5 concentric layers. Layers 1, 3, and 5 utilize the contrasting matte matMetalloChiaro accent color , while layers 2

and 4 apply the main matRobot PBR texture. A double-plated support bracket anchors a horizontal cylindrical main pivot. When it rotates left or right, it carries the rest of the arm with it.

- **Lower Arm:** Attached directly to the base assembly pivot. It consists of two parallel vertical rectangular plates and terminates with a central horizontal cylindrical joint articulation (acting as the mechanical pivot connecting the two arms of the robot), controlling the first tilt movement.
- **Upper Arm:** It features a central solid rectangular, a wrist mounting bracket and a cylindrical wrist joint. Being attached to the central joint it's moving along with it but allowing an independent secondary tilt.
- **Wrist & Claw:** Act as the end group. It contains two side plates, a circular base, and a central hexagonal form which consists in the hand that grab the ball itself.
- **Gripper Fingers:** Three symmetrical finger groups spaced 120 degrees apart around the vertical Y-axis. Each finger mesh is modeled via a partial torus initialized with the matMetalloChiaro color to make the mechanical grabber visually stand out from the rest of the dark arm structure.

Thanks to this hierarchy, when a joint rotates, the system automatically updates the position of all the attached child pieces, keeping the robot structure together naturally. The other models used in the scene are the spheres, the balls that the user has to capture, and the table which is a simple box with a grid over it. All of them, as can be seen in the code, are made using the library three.js that made the operations easier.

4.3. *Material and textures*

To change the appearance of the game based on the selected theme, all 3D objects use a MeshStandardMaterial. Different textures are used all across the project and on different elements of the scene, such as the surface grid, the spheres and the robotic arm. For the robotic arm and the grid surface is used one single texture for each of them which is a metal one for both cases, but the one of the arm is simpler while the table one has more finer details, with different applied colour in order to make them more coherent with the selected theme. The arm has the same texture across all pieces but with an applied grey-metal color instead of the one of the texture, to give a more realistic and complex look to the model. The spheres instead has three different textures, one for each theme, for the same reason as before: a marble one for the cyberpunk theme, a metal one for the industrial and an onyx one for the retrowave. In line with the project requirements, the PBR(Physically based rendering) material, which uses a package of more, separated images to map a singular texture and the physical details, allows me to apply the texture images to the surfaces to achieve a realistic look:

- **Color Map:** This defines the basic color, patterns, and visual skin of the objects (also known as Albedo). For the robotic arm and the grid surface, the code merely applies a

different color overlay at runtime to match the theme. For the target spheres, instead, this map is completely swapped along with its corresponding normal and roughness assets to transition between entirely different materials (marble, metal, and onyx) depending on the active layout.

- **Normal Map:** This simulates fine surface details, like scratches, metallic panels, and small bumps. This fulfills the explicit requirement for detail mapping, making the surfaces react realistically to the light sources without modifying the actual geometric complexity of the 3D model.
- **Specular Map:** This controls the surface shininess and reflectivity (mapped via the roughness/metalness channels in the standard material). It dictates which parts of the robotic arm and environment reflect lights sharply and which areas remain matte and diffused, matching the mandatory texture requirements.

4.4. Animations

All the animations in the project are computed procedurally at runtime without importing pre-baked external tracks as needed by the requirements, keeping the execution dynamic and interactive. They are written in javascript using three.js and tween.js to make them smoother, used to make the game more realistic and obviously making it even more dynamic, in the following cases:

- **Arm Movements:** The rotations of the robot joints are directly bound to keydown event listeners. When the user presses the control keys, the base rotates left and right around its vertical Y-axis. The lower arm and upper arm instead tilt up and down by rotating around their local horizontal Z-axis. By combining these rotations, the user can reach any point on the table without both the wrist and joint being able to go beyond the surface because real-time software monitors the global height of the claw and the central joint to prevent the arm from clipping through the table surface.
- **Claw Animation:** While the arm joints follow the direct keyboard inputs, the opening and closing of the claw is managed by Tween.js to transition the finger angles smoothly, making the grabbing action look fluid and natural when a ball is caught. When triggered, the three fingers rotate simultaneously and symmetrically around their local X-axes moving inward to close the gripper around a ball, or outward to release it back into the starting state. When a ball is being caught by the claw all the spheres disappear simultaneously by reducing its radius and then regenerated.
- **Collision Detection:** The code constantly calculates the straight-line distance between the center of the claw and each moving sphere. If this distance is smaller than their combined size, a collision is detected, and a point is scored. When a ball is being caught the animation of the three spheres disappearing simultaneously is handled in three states:
 - **Attrazione:** The sphere uses fractional linear interpolation to fly straight into the claw's global coordinates.

- ***Incollata***: Holds the sphere rigidly at the claw's center for a 300ms magnetic capture interval.
- ***Assorbimento / Sparizione***: The captured sphere shrinks while following the moving wrist, whereas uncaptured balls lock their positions on the table and fade away simultaneously. Scale is reduced logarithmically until it drops below 0.05, removing the mesh from the scene and triggering a new board spawn.
- **Table Boundaries**: The spheres translate across the table automatically. When a ball hits the mathematical edge of the play area, its direction vector is flipped, making it bounce back inside the workspace realistically.

5. Projectual choices

This final section outlines the fundamental decisions made to satisfy the project's optional requirements, focusing on user interaction, gameplay customization, and visual features. I want to go into more detail about the decisions made during the project's development, specifically those left to personal discretion, to clear even more all the aspects of the project.

5.1. Procedural vs. Imported 3D Models

Instead of importing pre-made 3D models from external files (such as .glTF or .obj), all the components of the scene, including the table, the room, and the robotic arm, were generated procedurally using the built-in geometries provided by Three.js (cylinders and boxes).

- **Why this choice**: This approach ensures that the application remains extremely lightweight and loads instantly, as it does not require downloading heavy asset files. Furthermore, building the robot parts mathematically via code allowed for a perfect alignment of the hierarchical pivot points, making the kinematic chain movements precise and easier to control.

5.2. Custom JavaScript vs. External Physics Engine

The simulation does not rely on a complex external physics engine (like Ammo.js or Cannon.js) to manage the ball movements and interactions. Instead, all collision detections and boundary responses are computed using custom JavaScript logic.

- **Why this choice**: Since the gameplay only requires checking the distance between the claw and the spheres, and managing simple linear bounces off the table edges, a full physics engine would have introduced a large amount of computational overhead. The custom distance-based collision system keeps the application performant and highly responsive.

5.3. Implementing Optional Advanced Lighting and Real-Time Shadows

Another free option for the project was the choice of the lightning setup, where the only constraint was that there is at least one light. I choose to include three different types of lights and real time shadows in order to make the scene look more complex, deeper ad realistic.

- **Different types of lights:** I used an ambient light, necessary for the project to keep the background, the robot and everything visible. Then I choose to include a pointlight on the top of the scene, above the arm to make this latter more visible and draw the user's attention, since their gaze will be drawn first to this, which is the main object of the game. In addition to these, I wanted the game too look more futuristic and modern so I used the pointlights, making them as neon tube, to highlight the bottom surface and the vertical corners of it.
- **Real-Time Shadow Mapping:** The directional light is configured to project dynamic shadows of the robotic arm and the balls directly onto the table surface. This choice makes the gameplay better because it helps the user understand the depth, the height, and the exact distance of the claw from the moving spheres other than obviously making the scene more realistic and complete (if there are lights the user will expect shadows).
- **Reflections and Materials:** By tweaking the roughness and metalness parameters of the MeshStandardMaterial for each theme, I created realistic environmental reflections. For example, the robot has high-contrast reflections to look like polished metal, while the table surface diffuses the light more softly.

5.4. User Interaction and UI Customization (Themes and Difficulty)

To address the user interaction requirements, I developed a changing theme and a difficulty selector. The display overlay allows the player to completely customize the environment and the gameplay loop in real time through these two main interactive features:

- **Theme Swapping:** The user can switch between three visual presets (Cyberpunk, Industrial, and Retrowave). This interaction triggers a dynamic update of the environment without reloading the page, swapping the color filters, the point light colors, and the texture maps of the spheres (marble, metal, and onyx) instantly.
- **Difficulty Settings:** The player can scale the difficulty between Easy, Medium, and Hard. Instead of changing a simple speed variable, this choice modifies multiple parameters dynamically at runtime including the target spheres' radius, their translation speed vectors, and the total count of active objects on the board, creating a progressive challenge based on user preference.

6. Project Architecture and Files

The project structure is kept minimal and organized to ensure clean separation between logic, styling, and assets. It consists of the following components:

- **index.html:** The entry point of the application. It sets up the WebGL canvas container and defines the HUD (Heads-Up Display) overlay, including the control panel for themes, difficulty levels, and the game instructions layout.
- **main.js:** The core JavaScript file containing the entire application logic. It handles the Three.js scene initialization (camera, lights, renderer), builds the hierarchical robot model, loads the PBR textures, and manages the real-time loop for movements, Tween.js claw animations, and collision detection.
- **style.css:** Contains the styling for the user interface. It handles the absolute positioning of the HUD panels, custom typography (monospace for an arcade feel), and responsiveness.
- **/textures and /libs:** External directories used to store the asset image packages (Color, Normal, and Specular maps for each theme) and the local library files for Three.js and Tween.js.